

API Reference: LED & Button Control

`xm_api_led_btn.h`에 정의된 **내장 UI(LED, Button) 제어 API**에 대한 상세 레퍼런스입니다. XM10 펌웨어는 사용자가 복잡한 타이머 인터럽트나 디바운싱(Debouncing) 로직을 직접 구현할 필요 없이, 직관적인 함수 호출만으로 **깜빡임(Blink)**, **클릭(Click)**, **롱 프레스(Long Press)** 등의 고급 입출력 기능을 구현할 수 있도록 돕습니다.

1. 동작 원리 (Operating Principle)

이 모듈은 시간 기반(Time-driven)으로 동작하며, 이를 위해 시스템 내부의 **Manager**들이 백그라운드에서 열심히 일하고 있습니다.

The Core Engine: `XM_IO_Update()`

- **역할:** LED의 깜빡임 타이밍을 계산하고, 버튼의 신호를 분석하여 노이즈를 제거(Debouncing)하고 이벤트를 생성하는 **엔진**입니다.
- **구동:** `core_process`에 의해 **2ms(500Hz)** 주기로 자동 호출되므로, 사용자가 직접 호출할 필요는 없으나 그 존재와 역할은 이해하고 있어야 합니다.
- **흐름:**
 1. 사용자가 `XM_SetLedEffect` 호출 -> 설정값 저장.
 2. `XM_IO_Update` 주기적 실행 -> 시간이 흐름에 따라 LED ON/OFF 토글.
 3. 사용자가 버튼 누름 -> `XM_IO_Update`가 시간 측정 -> '클릭' 또는 '롱 프레스' 판단 후 이벤트 큐에 저장.
 4. 사용자가 `XM_GetButtonEvent` 호출 -> 큐에서 꺼내서 반환.

2. 데이터 구조 (Enumerations)

사용 편의성을 위해 모든 식별자와 상태 값은 **Enum**으로 정의되어 있습니다. 숫자(0, 1, 2...) 대신 이 상수를 사용하세요.

2.1. Identifiers (식별자)

`XmLedId_t`

제어할 Function LED의 번호입니다.

```
typedef enum {  
    XM_LED_1 = 1, /**< 왼쪽 LED */  
    XM_LED_2 = 2, /**< 가운데 LED */  
    XM_LED_3 = 3  /**< 오른쪽 LED */  
} XmLedId_t;
```

`XmBtnId_t`

입력을 확인할 Function Button의 번호입니다.

```
typedef enum {
    XM_BTN_1 = 1, /**< 왼쪽 버튼 */
    XM_BTN_2 = 2, /**< 가운데 버튼 */
    XM_BTN_3 = 3 /**< 오른쪽 버튼 */
} XmBtnId_t;
```

2.2. State Types (상태 정의)

XmState_t

LED 등의 논리적인 On/Off 상태를 표현합니다.

```
typedef enum {
    XM_OFF = 0, /**< 끄기 (Logic Low) */
    XM_ON = 1 /**< 켜기 (Logic High) */
} XmState_t;
```

XmBtnState_t

버튼의 현재 물리적 눌림 상태를 표현합니다. (회로의 Active-High/Low 여부와 상관없이 추상화됨)

```
typedef enum {
    XM_RELEASED = 0, /**< 버튼이 떨어져 있음 (안 눌림) */
    XM_PRESSED = 1 /**< 버튼이 눌러 있음 */
} XmBtnState_t;
```

2.3. Modes & Events (모드 및 이벤트)

XmLedMode_t

LED의 동작 패턴을 결정합니다.

Mode	Description
<code>XM_LED_OFF</code>	LED를 끕니다.
<code>XM_LED_SOLID</code>	LED를 계속 켭니다.
<code>XM_LED_BLINK</code>	일정 주기(Period)로 깜빡입니다. (50% Duty)
<code>XM_LED_HEARTBEAT</code>	심장 박동처럼 두 번 빠르게 깜빡입니다. (두근-두근)
<code>XM_LED_ONESHOT</code>	설정된 시간만큼 한 번 켜졌다가 자동으로 꺼집니다. (알림용)

XmBtnEvent_t

버튼 조작을 분석하여 생성된 고수준 이벤트입니다.

Event	Description
<code>XM_BTN_NONE</code>	발생한 이벤트 없음
<code>XM_BTN_PRESSED</code>	버튼을 막 누른 순간 (Rising Edge)
<code>XM_BTN_RELEASED</code>	버튼을 막 떼는 순간 (Falling Edge)
<code>XM_BTN_CLICK</code>	짧게 눌렀다 떼م (클릭)
<code>XM_BTN_LONG_PRESS</code>	1초 이상 길게 누름 (모드 변경 등에 사용)

3. 함수 상세 (Function Reference)

3.1. LED Control Functions

`XM_SetLedState`

LED를 단순히 켜거나 끕니다. 가장 직관적인 제어 함수입니다.

- **Syntax**

```
void XM_SetLedState(XmLedId_t led_id, XmState_t state);
```

- **Parameters**

- `led_id`: 제어할 LED (`XM_LED_1 ~ 3`)
- `state`: 목표 상태 (`XM_ON`, `XM_OFF`)

- **Example**

```
// 1번 LED 켜기
XM_SetLedState(XM_LED_1, XM_ON);
```

`XM_SetLedEffect`

LED에 특수 효과(깜빡임 등)를 설정합니다. 설정 후에는 `XM_IO_Update()`에 의해 자동으로 동작하므로, 별도의 타이머 코드를 작성할 필요가 없습니다.

- **Syntax**

```
void XM_SetLedEffect(XmLedId_t led_id, XmLedMode_t mode, uint32_t period_ms);
```

- **Parameters**

- `led_id`: 제어할 LED

- **mode**: 동작 모드 (`XM_LED_BLINK`, `XM_LED_HEARTBEAT` 등)
- **period_ms**: 효과의 주기 또는 지속 시간 (밀리초 단위)
 - **BLINK**: 깜빡임 주기 (예: 500 = 0.5초 간격)
 - **ONESHOT**: 켜져 있는 시간 (예: 200 = 0.2초간 점등 후 소등)
 - **SOLID/OFF**: 무시됨 (0 입력 권장)

- **Example**

```
// 2번 LED를 0.5초 간격으로 깜빡이게 설정
XM_SetLedEffect(XM_LED_2, XM_LED_BLINK, 500);

// 3번 LED를 심장 박동 모드로 설정 (1초 주기)
XM_SetLedEffect(XM_LED_3, XM_LED_HEARTBEAT, 1000);
```

3.2. Button Control Functions

`XM_GetButtonState`

버튼의 **현재 물리적 상태**를 실시간으로 확인합니다. (Polling 방식)

- **Syntax**

```
XmBtnState_t XM_GetButtonState(XmBtnId_t btn_id);
```

- **Parameters**

- **btn_id**: 확인할 버튼 (`XM_BTN_1 ~ 3`)

- **Returns**: `XM_PRESSED` (눌림) 또는 `XM_RELEASED` (안 눌림)

- **Example**

```
// 버튼 1을 누르고 있는 동안만 실행
if (XM_GetButtonState(XM_BTN_1) == XM_PRESSED) {
    // ... 동작 수행 ...
}
```

`XM_GetButtonEvent`

버튼에서 발생한 **최신 이벤트**를 가져옵니다. (Event-Driven 방식) 이벤트를 한 번 읽으면 내부 큐에서 사라집니다 (**Read-Clear**). 따라서 변수에 저장해서 사용하거나 `switch` 문에서 사용하는 것이 좋습니다.

- **Syntax**

```
XmBtnEvent_t XM_GetButtonEvent(XmBtnId_t btn_id);
```

- **Parameters**
 - `btn_id`: 확인할 버튼
- **Returns**: 감지된 이벤트 (`XM_BTN_CLICK`, `XM_BTN_LONG_PRESS` 등)
- **Example**

```
XmBtnEvent_t evt = XM_GetButtonEvent(XM_BTN_1);

if (evt == XM_BTN_CLICK) {
    // 버튼 1이 '딸깍' 클릭되었을 때 1회 실행
    SendUsbDebugMessage("Button 1 Clicked!\r\n");
}
else if (evt == XM_BTN_LONG_PRESS) {
    // 버튼 1이 '꾸욱' 눌렀을 때 1회 실행
    SendUsbDebugMessage("Button 1 Long Pressed!\r\n");
}
```

3.3. System Functions

`XM_IO_Update`

[시스템 필수] LED와 버튼의 상태를 업데이트하는 구동 함수입니다. User Task의 무한 루프(`User_Loop` 또는 `TSM_Run` 내부)에서 주기적으로 호출되어야 `Blink`나 `Long Press` 기능이 정상 동작합니다.

- **Syntax**

```
void XM_IO_Update(void);
```

- **Note**: `core_process.c`가 `_FetchAllInputs` 과정에서 자동으로 호출해주므로, 일반적인 경우 **End User**가 직접 호출할 필요는 없습니다.